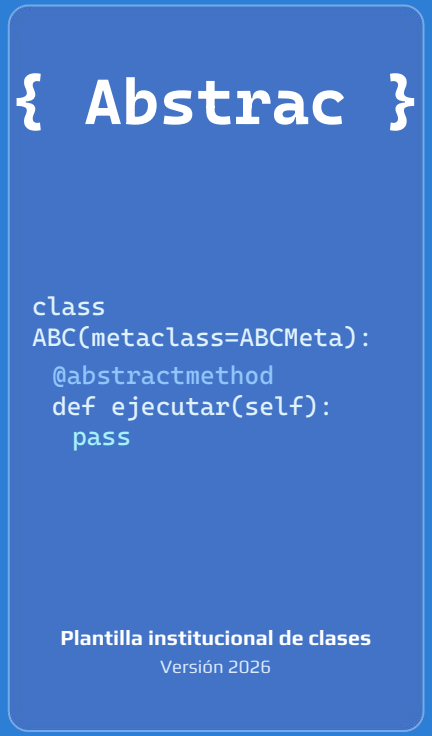
DAO

Clases y métodos Abstractos

Python • POO • Clases Abstractas • Interfaces

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA



CONTENIDOS

Contenido

01

¿Qué es?

Concepto clave de la programación orientada a objetos y su flexibilidad.

02

Clases Abstractas

Polimorfismo estructural y definición de contratos formales de diseño.

03

Ejemplo Práctico

Implementación guiada de código real en lenguaje Python.

04

Conclusiones

Análisis de beneficios, acoplamiento mínimo y mejores decisiones de arquitectura.

¿Qué es el Polimorfismo? 🤔

Definición

"La habilidad de un objeto de tomar múltiples formas." Permite que diferentes objetos respondan al mismo mensaje de manera distinta.

Analogía práctica

Un mismo comando, **"sonar"**, puede ser interpretado de forma diferente por cada receptor:

- **Perro:** responde con un ladrido.
- **Gato:** responde con un maullido.



Polimorfismo de 'Duck Typing'

¿Qué es el Duck Typing?

En Python, el polimorfismo se basa en este concepto: "Si parece un pato y camina como un pato, entonces es un pato".

No importa el tipo o la clase del objeto, sino **lo que puede hacer** (sus métodos y comportamiento).

Ejemplo práctico:

Una función que invoca al método `vuela()` puede procesar de igual manera un objeto de la clase **Pájaro** o de la clase **Avión**.



¿Qué es una Clase Abstracta? 🧩

Concepto Base

Es una **clase incompleta** que no se puede instanciar por sí misma. Su propósito principal es servir como molde o plantilla estructural.

Propiedades clave

- **Clase incompleta:** Diseñada exclusivamente para ser heredada.
- **Métodos comunes:** Puede contener lógica compartida ya implementada.
- **Estructura:** Define la interfaz que las subclases deben cumplir.

Tres pilares fundamentales:

1. Plantilla (Molde)

Proporciona una guía estructural y lógica común para todas las clases que hereden de ella.

2. Contrato (Definición)

Define explícitamente qué métodos deben existir en las clases hijas, garantizando consistencia.

3. Implementación Forzada

Obliga legalmente a que cada subclase concreta defina el comportamiento de los métodos abstractos.

¿Qué es un Método Abstracto?

Definición del Concepto

Es un método **declarado** en una clase abstracta, pero que **no posee implementación**. Define un contrato obligatorio para sus subclases.

Reglas y Aplicación:

- **Sobreescritura Obligatoria:** Debe ser implementado obligatoriamente por todas las clases hijas que hereden de la clase abstracta.
- **Uso de Decorador:** Se define utilizando el decorador `@abstractmethod` del módulo **`abc`** (Abstract Base Classes).

Módulo 'abc' en Python

"Python utiliza el módulo **abc** para definir clases y métodos abstractos."

Pasos clave:

1. **Importar:** Obtener `ABC` y `abstractmethod` de `abc`.
2. **Heredar:** La clase padre debe heredar de la clase `ABC`.
3. **Decorar:** Marcar métodos obligatorios con `@abstractmethod`.

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def hacer_sonido(self):
        pass

class Perro(Animal):
    def hacer_sonido(self):
        print("Guau!")

class Gato(Animal):
    def hacer_sonido(self):
        print("Miau!")
```

Polimorfismo y Abstracción

Abstracción

"La abstracción define el qué (la interfaz)."

Polimorfismo

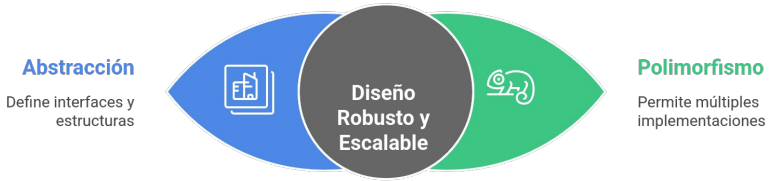
"El polimorfismo maneja el cómo (la implementación)."

Integración

"Ambos conceptos trabajan juntos para crear diseños más robustos y escalables."

Sinergia de Diseño

El Poder de la Abstracción y el Polimorfismo Combinados



CASO PRÁCTICO

Diseño de Sistema Asistido

El Problema

Debemos diseñar un sistema de diseño asistido que debe ser capaz de representar figuras geométricas. El sistema debe poder ser extensible mediante plugins.

Requisitos de cada Plugin:

- **Inicialización:** Se debe poder crear indicando el nombre y la longitud de sus lados.
- **Propiedades:** Debe poder indicar su nombre, su superficie y su perímetro.

Solución con Herencia y Polimorfismo

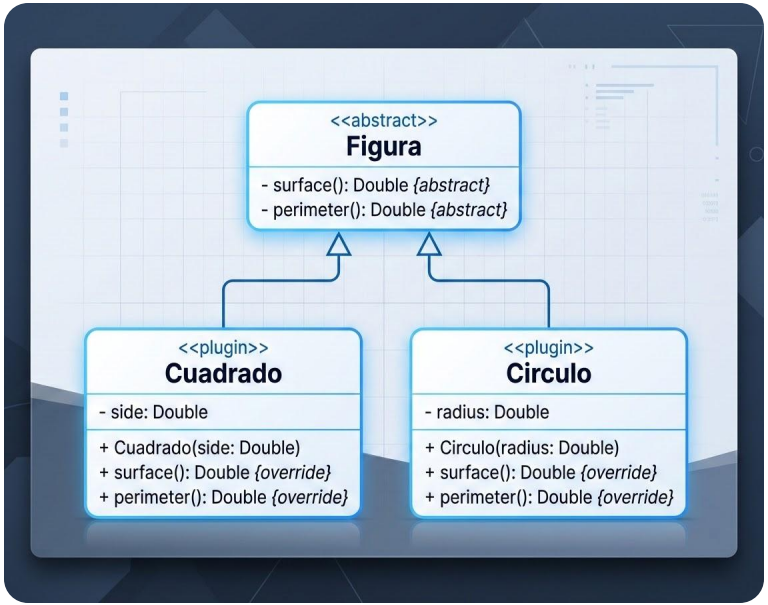
Estructura del Contrato

Se define una clase abstracta base `Figura` que actúa como plantilla y contrato para todos los plugins.

Mecanismo clave:

1. Herencia: Cada plugin de figura (`Circulo`, `Cuadrado`) hereda de la clase abstracta base `Figura`.

2. Polimorfismo: El sistema asistido procesa una lista de objetos `Figura` llamando a `superficie()` y `perimetro()` sin conocer su tipo concreto.



¡Gracias!

Hasta la próxima

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA